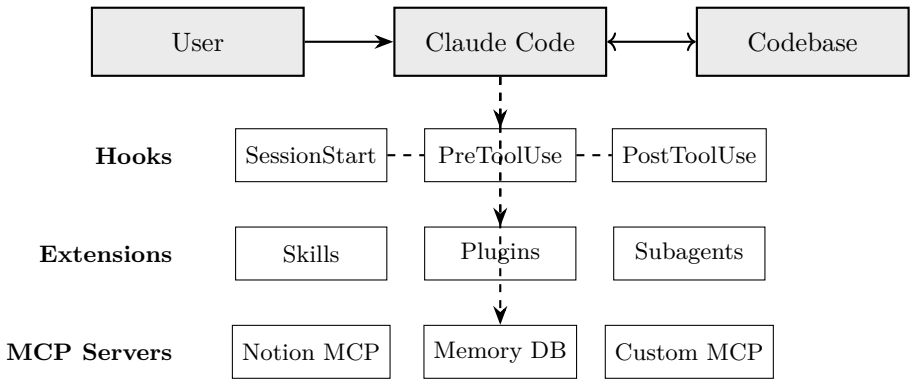
The Claude Code Ecosystem

Survey, Evaluation & Implementation Guide

A comprehensive review of 29 repositories, tools and frameworks



29 repositories · 8 domains · 1,500+ skills surveyed

6 key sources deep-absorbed · 3 rounds adversarial review

Yggdra · Personal Knowledge System

March 2026

Contents

1 Abstract **4**

2 Method **5**
2.1 Research Process . . . 5
2.2 Limitations . . . 5

3 Core Concepts — Deep Dive **6**
3.1 Claude Code . . . 6
3.2 The Context Window . . . 6
3.3 Context Rot . . . 6
3.4 Skills . . . 7
3.5 Hooks . . . 7
3.6 MCP (Model Context Protocol) . . . 8
3.7 Subagents . . . 9
3.8 Token Economics . . . 9

4 Repository Catalog — Official Anthropic **10**
4.1 anthropics/skills (86.9k stars) . . . 10
4.2 anthropics/claude-code (75.1k stars) . . . 10
4.3 anthropics/claude-cookbooks (34.4k stars) . . . 11
4.4 anthropics/courses (19.2k stars) . . . 11
4.5 anthropics/claude-quickstarts (15.1k stars) . . . 11
4.6 anthropics/claude-code-action (6.1k stars) . . . 12
4.7 anthropics/claude-agent-sdk-python (5.2k stars) . . . 12

5 Repository Catalog — Community Skills & Memory **13**
5.1 affaan-m/everything-claude-code [ECC] (65.8k stars) . . . 13
5.2 thedotmack/claude-mem (33.4k stars) . . . 14
5.3 alirezarezvani/claude-skills (2.3k stars) . . . 15
5.4 Jeffallan/claude-skills (5.6k stars) . . . 15
5.5 sickn33/antigravity-awesome-skills (21.6k stars) . . . 15
5.6 VoltAgent/awesome-claude-code-subagents . . . 16
5.7 makenotion/claude-code-notion-plugin . . . 16
5.8 YouMind-OpenLab/nano-banana-pro-prompts . . . 16
5.9 tfriedel/claude-office-skills . . . 16
5.10 mksglu/context-mode (2.9k stars) . . . 17

6 Repository Catalog — Workflow & Architecture **18**
6.1 gsd-build/get-shit-done [GSD] (26.1k stars) . . . 18
6.2 ChrisWiles/claude-code-showcase (5.5k stars) . . . 19
6.3 NicholasSpisak/claude-code-subagents . . . 19
6.4 shanraisshan/claude-code-best-practice . . . 20
6.5 teambrilliant/claude-research-plan-implement [RPI] . . . 20
6.6 dontriskit/awesome-ai-system-prompts . . . 20
6.7 lbjlaq/Antigravity-Manager . . . 21
6.8 alibaba/page-agent . . . 21

7 Repository Catalog — Visual Tools **22**
7.1 mermaid-js/mermaid (86.5k stars) . . . 22

7.2 mingrammer/diagrams (42.1k stars) . . . 22
7.3 antvis/Infographic (4.6k stars) . . . 23
7.4 federicodeponte/opendraft (55 stars) . . . 23
7.5 Shubhamsaboo/awesome-llm-apps (100k stars) . . . 23
7.6 openclaw/openclaw . . . 23

8 Gap Analysis — Yggdra vs. The Ecosystem **25**
8.1 What Yggdra already has (validated by the ecosystem) . . . 25
8.2 What Yggdra is missing (prioritized) . . . 25
8.3 What Yggdra does better . . . 26

9 Recommendations **27**
9.1 Classification summary . . . 27
9.2 Implementation plan . . . 27
9.2.1 Step 1: Foundation . . . 27
9.2.2 Step 2: Official plugins + hooks . . . 27
9.2.3 Step 3: Community trials . . . 27
9.2.4 Step 4: Workflow . . . 28
9.2.5 Step 5: Memory . . . 28
9.2.6 Ongoing . . . 28

10 Maintenance & Kill Conditions **29**
10.1 Cadences . . . 29
10.2 Kill conditions . . . 29
10.3 Cost monitoring . . . 29

11 Open Questions **30**

12 Known Issues **30**

13 Complete Link Catalog **31**
13.1 Official Anthropic . . . 31
13.2 Community — Skills & Memory . . . 31
13.3 Workflow & Architecture . . . 31
13.4 Visual Tools . . . 31
13.5 Guides & Articles . . . 31

1 Abstract

Claude Code is Anthropic's official command-line interface for working with Claude directly in the terminal or an IDE. Rather than using a browser chat, you type in your terminal and Claude reads files, edits code, runs commands, and works across entire projects. Since its launch, a full ecosystem has emerged around it—skills, plugins, hooks, memory systems, workflow frameworks, and visual tools—created both by Anthropic and by the community.

This report surveys that ecosystem as of March 2026. 29 repositories and sources have been reviewed across 8 domains. Each repository is evaluated individually: what it does, when it fits, what caveats to consider, and whether better alternatives exist. The report provides deep technical explanations of core concepts (context windows, hooks, MCP, subagents, context rot), a gap analysis against an existing production system (Yggdra), and a concrete implementation plan.

Key findings:

- The ecosystem is dominated by a small number of high-quality repositories (ECC, GSD, claude-mem) that have established patterns now imitated by hundreds of smaller projects.
- **Progressive disclosure**—retrieving information in layers rather than all at once—is the single most impactful innovation for token efficiency.
- **Hooks** (automatic triggers on events) are more reliable than skills (instructions Claude chooses to follow) for critical workflows.
- Most repositories solve one of three problems: memory persistence, workflow structure, or token optimization. Very few address all three.
- The official Anthropic plugins are conservative but reliable. Community tools are more ambitious but carry dependency and maintenance risks.

2 Method

2.1 Research Process

The report was produced in five phases:

Phase 1: Broad survey
Systematic review of GitHub repositories via `gh` CLI and WebFetch. Sources found through: Anthropic official repos, community awesome-lists, Hacker News discussions, YouTube. All raw data saved in `research-dump-claude-code-ecosystem.md` (363 lines). Grouped into 8 domains.

Phase 2: Deep absorption
5 parallel agents absorbed 6 key sources. Each agent had specific extraction goals: read README, source code, config files, templates. Extract schemas, hooks, workflows, patterns. 1 agent stalled and was manually replaced (ISS-001).

Phase 3: Evaluation
Each tool classified using the ThoughtWorks Technology Radar model with 4 criteria: maturity (stars, community activity, maintenance cadence), relevance to Yggdra, cost/risk (license terms, dependencies, token overhead), and overlap with existing tools.

Phase 4: Adversarial review (3 rounds)
Red team attacks recommendations and finds weaknesses. Blue team defends or concedes. Neutral judge implements changes. 9 corrections from this process.

Phase 5: Self-critique (3 rounds)
Structural critique (missing sections?), content critique (are claims substantiated?), usability critique (can a reader act on this?). 8 additional fixes.

2.2 Limitations

- **Star counts** indicate popularity, not validated quality.
- **Token savings claims** (e.g. claude-mem's "∼10x") are from the sources' own benchmarks, not independently verified.
- **Snapshot in time:** All data is from March 8, 2026. The ecosystem evolves rapidly.
- **Selection bias:** Only English-language, GitHub-hosted repos are included.

3 Core Concepts — Deep Dive

3.1 Claude Code

Claude Code is a terminal program. You start it with the command `claude` and then write in natural language—"find all TODO comments in src/" or "add a login page." Claude reads files, edits code, runs tests, and navigates the entire project. It differs from a browser chat because Claude has direct access to the filesystem and terminal.

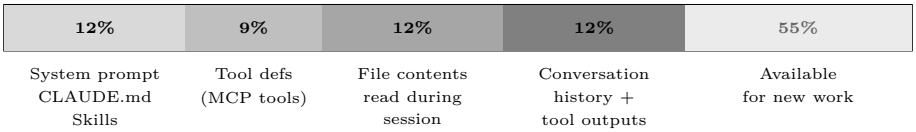
Claude Code can also run inside an IDE (e.g. VS Code) as a sidebar panel, so you see code and Claude's work side by side. All skills, hooks, and MCP servers work identically in both modes.

What Claude Code is not: It is not an autonomous agent that runs indefinitely. It responds to prompts, uses tools, and stops. Autonomy must be built on top—via hooks, cron jobs, or external orchestration.

3.2 The Context Window

The context window is the total amount of information Claude can "see" at once. As of March 2026, it is 200,000 tokens (≈150,000 words). This sounds like a lot, but it fills quickly:

200K token context window



These percentages are approximate and shift during a session. The key insight: **before Claude does any real work, 30–50% of the window may already be consumed** by system prompt, tool definitions, and loaded files. With too many MCP servers, the available space shrinks dramatically—ECC reports that 200K can effectively become ∼70K with excessive tooling.

Token costs (Opus 4.6): Input tokens cost $15 per million; output tokens cost $75 per million. A session consuming 200K tokens costs approximately $3. Token efficiency is therefore both a quality concern (more room for useful content) and a financial one.

3.3 Context Rot

Context rot is the degradation of Claude's output quality over the course of a long session. It happens because:

1. The context window fills with file contents, tool outputs, error messages, and conversation history.
2. When the window approaches capacity, Claude Code **automatically compresses** the older portion of the conversation. This is called "compaction."
3. Compaction is lossy—details from earlier in the session are summarized or dropped entirely.
4. Claude may "forget" decisions made earlier, repeat mistakes, or lose track of the overall plan.

Symptoms of context rot:

- Claude asks questions that were already answered
- Code changes conflict with earlier changes in the same session

- Claude loses track of which files it has already modified
- Output quality drops noticeably compared to the start of the session

Mitigation strategies (explored in detail in later sections):

- **Fresh context per task** (GSD pattern): spawn a new subagent for each subtask
- **Context monitor hooks**: warn at 35% remaining, halt at 25%
- **Strategic compaction**: manually compact at logical breakpoints
- **State files**: save progress to disk so it survives compaction

3.4 Skills

A **skill** is a reusable instruction for Claude Code, stored as a markdown file in the `.claude/skills/` directory. When Claude encounters a task matching the skill's description, it follows the instructions in the file.

Example: A skill called "commit-message" contains rules for how commit messages should be formatted in a specific project. Every time Claude makes a commit, it follows those rules automatically.

Skills can also be **user-invocable**—started manually with a slash command like `/commit` or `/review`. They can accept arguments (`/deploy staging`), run shell commands as preprocessing, and spawn subagents that work in isolation.

Key skill properties:

`context: fork` The skill runs in an isolated subagent with its own context. Results return to the main conversation. Useful for heavy tasks that shouldn't pollute the main context.

`allowed-tools` Restricts which tools the skill may use. A read-only skill can be limited to Read and Grep.

`model` Overrides the model. A cheap scan skill can use Haiku; an architecture skill can require Opus.

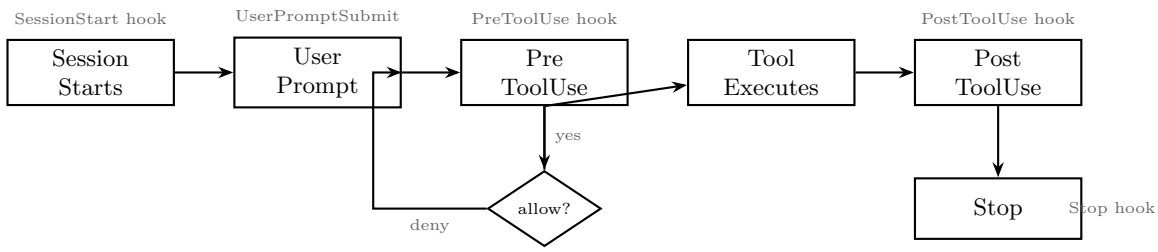
`!`command`**` Shell commands that run **before the skill is sent to Claude. Output replaces the placeholder. This is preprocessing, not runtime execution.

`$ARGUMENTS` Replaced with the user's input. `$ARGUMENTS[0]` for positional arguments.

Budget Skills may occupy at most 2% of the context window (∼16,000 characters as fallback). Override with `SLASH_COMMAND_TOOL_CHAR_BUDGET`.

3.5 Hooks

Hooks are automatic actions that execute when specific events occur in Claude Code. They are shell scripts (bash, Python, Node) executed by the system—not by Claude itself. This distinction matters: Claude "chooses" whether to follow a skill's instructions (and sometimes doesn't), but hooks fire with 100% reliability because they are hard triggers.



Hook output format: Hooks communicate with Claude Code via JSON on stdout:

```
{
  "hookSpecificOutput": {
```

```
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Edits on main blocked."
  }
}
```

There are **17 hook events** in total. The most important ones:

SessionStart Fires when a session starts. Used to load context, previous state, or inject instructions. Matchers: `startup`, `resume`, `clear`, `compact`.

UserPromptSubmit Fires when the user sends a message. Can **modify** the message before Claude sees it. Used to add context or redirect.

PreToolUse Fires before Claude uses a tool (Edit, Bash, Write, etc.). Can **block** the tool. Used for safety—e.g. preventing edits on the main branch. Matchers: tool name regex.

PostToolUse Fires after a tool succeeds. Used for auto-formatting, auto-testing, or data collection. Non-blocking.

PreCompact Fires before Claude compresses its context. Used to save important state before it is lost.

Stop Fires when Claude finishes its response. Used for checkpoints.

SubagentStart / SubagentStop Fires when subagents spawn or finish. Used for logging and rate-limiting.

Hook priority order: Managed policy > `~/.claude/settings.json` > `.claude/settings.json` > `.claude/settings.local.json` > Plugin hooks > Skill frontmatter hooks.

3.6 MCP (Model Context Protocol)

MCP is a standard for giving Claude Code access to external systems. An **MCP server** is a program that exposes tools which Claude can call. For example, a Notion MCP server runs locally and gives Claude tools like "search pages," "create page," and "update page."

MCP servers are configured in `.mcp.json` (project-level) or `.mcp.json.local` (machine-specific):

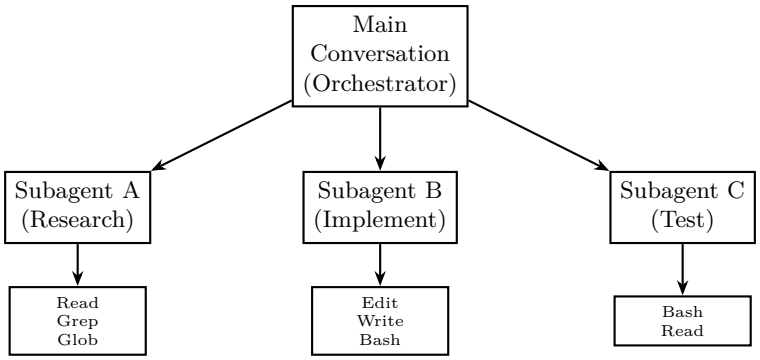
```
{
  "mcpServers": {
    "notion": {
      "command": "npx",
      "args": ["-y", "@notionhq/notion-mcp-server"],
      "env": {"OPENAPI_MCP_HEADERS": "..."}
    }
  }
}
```

Critical insight: Every tool an MCP server exposes consumes tokens in the context window just by existing (the tool definition must be present for Claude to know it is available). This means **fewer MCP servers = more space for actual work**. ECC recommends a maximum of 10 MCP servers and 80 active tools. Beyond that, the effective context window shrinks dramatically.

The alternative to always-loaded MCP servers is **CLI-wrapped skills**: shell commands wrapped in a skill that only loads when invoked. This uses zero tokens when idle.

3.7 Subagents

A subagent is a separate Claude instance spawned by the main conversation. It gets its own context (typically a fresh 200K window) and works in isolation on a subtask. When finished, it returns the result to the main conversation.



Trade-offs:

- Each spawn costs ∼200K tokens (≈$0.60–1.00) before the subagent does any work.
- The subagent has no access to the main conversation's history—it starts fresh.
- Multiple subagents can run in parallel, which is powerful for independent tasks.
- Coordination between subagents requires explicit design (shared files, state on disk).

In skills, use `context: fork` to spawn a subagent. In conversation, use the Agent or Task tool.

3.8 Token Economics

A summary of costs relevant to ecosystem tool choices:

Item	Description	Approx. cost
Opus 4.6 input	Per 1M tokens	$15
Opus 4.6 output	Per 1M tokens	$75
Sonnet 4.6 input	Per 1M tokens	$3
Sonnet 4.6 output	Per 1M tokens	$15
Haiku 4.5 input	Per 1M tokens	$0.80
Haiku 4.5 output	Per 1M tokens	$4
Typical Opus session	200K tokens mixed	∼$3–5
GSD subagent spawn	200K fresh context each	∼$0.60–1
5 parallel subagents	GSD wave execution	∼$3–5
ECC hooks overhead	Per tool call (standard)	∼500–2K tokens

Key principle: Every optimization that reduces token consumption—progressive disclosure, CLI-wrapped skills, strategic model selection—pays for itself both in cost savings and in better output quality (more room for useful context).

4 Repository Catalog — Official Anthropic

Each repository is evaluated on: what it does, when it fits, caveats, and whether better alternatives exist.

4.1 anthropics/skills (86.9k stars)

What it is: The official skills standard definition and reference implementation. This is the canonical source for how skills work: frontmatter syntax, lifecycle, discovery rules, and the `agentskills.io` specification.

What it contains: Documentation, schema definitions, and reference skills. Not a collection of ready-to-use skills—rather, the specification that all other skill repos follow.

When it fits: Always. If you build or install any skills, this is the authoritative reference.

Caveats: The specification evolves. Features like `context: fork` and `hooks` in frontmatter are relatively new. Pin to a version or check the changelog before relying on bleeding-edge features.

Alternatives: None. This is the standard.

4.2 anthropics/claude-code (75.1k stars)

What it is: The Claude Code CLI itself, plus **13 official plugins**. The repo contains both the core tool and first-party extensions.

The 13 plugins in detail:

hookify Creates hooks via natural language. Say "block edits on main" and it generates the correct hook configuration. Useful because writing hook JSON manually is error-prone.

security-guidance PreToolUse hook that checks for 9 security anti-patterns before Claude executes:

1. SQL injection in Bash commands
2. Hardcoded credentials/API keys in code
3. Unsafe deserialization
4. Command injection via user input
5. XSS in HTML/JS output
6. Path traversal in file operations
7. Insecure HTTP (vs HTTPS)
8. Over-privileged permissions
9. Sensitive data in logs/output

commit-commands Adds `/commit` and `/commit-push-pr`. The former creates well-formatted conventional commits. The latter goes the full distance: commit, push, create Pull Request with description.

feature-dev 7-phase development workflow: discover → plan → implement → test → review → commit → PR. Spawns multiple subagents at each phase.

code-review 5 parallel review agents analyze code from different angles (correctness, performance, security, readability, testing).

pr-review-toolkit Structured PR review with checklist. Less ambitious than code-review but more practical for daily use.

plugin-dev Helps build new plugins. Meta-tool.

ralph-wiggum Humorous response format. Not practically useful.

frontend-design Frontend-focused skill for UI development. Understands CSS frameworks, component libraries, responsive design.

explanatory-output-style Forces Claude to explain its reasoning. Good for learning; adds overhead for production work.
learning-output-style Teaching-friendly output with step-by-step explanations.
agent-sdk-dev Help for building with the Agent SDK.
claude-opus-4-5-migration Migration helper for Opus 4.5 code patterns.

When it fits: The three "must-install" plugins are hookify, security-guidance, and commit-commands. They have zero risk (official source), low overhead, and solve common needs. The others are situational.

Caveats: feature-dev and code-review spawn multiple subagents and can be expensive ($5–10 per invocation). Use them for important work, not routine changes.

4.3 anthropics/claude-cookbooks (34.4k stars)

What it is: Practical recipes from Anthropic. Step-by-step tutorials for common patterns.

Key recipes:

- **RAG (Retrieval-Augmented Generation):** How to combine search in your own data with Claude. Directly relevant for any Qdrant-based setup.
- **Tool use:** Designing custom tools and integrations.
- **PDF & Vision:** Working with documents and images.
- **Sub-agents:** Patterns for multi-agent workflows.
- **SQL & Data:** Structured data manipulation.

When it fits: When building new integrations or learning Claude's tool use patterns. Reference material, not installable.

Caveats: Some recipes may lag behind the latest API. Check dates.

Alternatives: claude-quickstarts for more complete starter projects.

4.4 anthropics/courses (19.2k stars)

What it is: Anthropic Academy—13 free courses with certificates. Course materials (Jupyter notebooks, slides) are in this repo.

Most relevant courses:

Introduction to Agent Skills Foundational understanding of skills. Prerequisite for everything else.
Claude Code in Action (1 session) Practical walkthrough of Claude Code features: hooks, skills, multi-file editing, subagents.
MCP Advanced Topics Depth on Model Context Protocol: custom servers, tool design, performance optimization.
Tool Use & Function Calling How Claude interacts with tools. Important for designing good skills and hooks.
Prompt Engineering for Developers Generally useful for writing better skills and CLAUDE.md instructions.

When it fits: For onboarding. Recommended order: Agent Skills → Claude Code in Action → MCP Advanced.

Caveats: Courses are introductory. They teach concepts, not production patterns.

4.5 anthropics/claude-quickstarts (15.1k stars)

What it is: Starter projects from Anthropic.

Key starters:

Autonomous Coding Agent Two-agent pattern: a planner and an executor. Planner breaks down the task; executor implements each step. Similar principle to GSD but simpler.

Computer Use Demo Claude controls a computer via screenshots + clicks/keystrokes.

When it fits: When building custom agent pipelines from scratch. The Autonomous Coding Agent pattern is a good starting point before adopting GSD.

Caveats: These are demos, not production-ready frameworks.

4.6 anthropics/claude-code-action (6.1k stars)

What it is: GitHub Actions integration for Claude Code. Run Claude as part of CI/CD pipelines.

When it fits: When you want automated code review on PRs, periodic documentation updates, or dependency audits—all powered by Claude. See claude-code-showcase for concrete workflow examples.

Caveats: Each GitHub Actions run consumes Claude API tokens. Set max-turns and timeout limits to control costs. A monthly docs-sync run with 30 turns can cost $5–10.

4.7 anthropics/claude-agent-sdk-python (5.2k stars)

What it is: Python SDK for building custom agents that use Claude as their "brain." The agent runs as a Python process and calls Claude's API for decisions.

When it fits: When building automation scripts that need Claude's reasoning—e.g. a research pipeline that reads papers, extracts findings, and organizes them. Or a monitoring agent that reads logs and takes action.

Caveats: Requires Python programming. The SDK handles conversation management and tool use, but you design the agent's architecture. Not a no-code solution.

Alternatives: For simpler automation, Claude Code + hooks + cron may be sufficient.

5 Repository Catalog — Community Skills & Memory

5.1 affaan-m/everything-claude-code [ECC] (65.8k stars)

What it is: The largest and most influential community collection. 65 skills, 16 agent configurations, 3 in-depth guides. But ECC's real contribution is not just skills—it is a **philosophy** for using Claude Code effectively.

Core principles:

- **Token optimization:** Keep MCP servers minimal. Replace always-loaded MCP with CLI-wrapped skills that only consume tokens when invoked.
- **Model selection:** Use Haiku for exploration/docs. Sonnet for 90% of coding. Opus only for bugs, 5+ file changes, architecture, or security.
- **Dynamic context injection:** `claude -system-prompt "$(cat memory.md)"` injects context with higher authority than CLAUDE.md.
- **Verification metrics:** pass@k (at least 1 of k attempts succeeds) vs pass^k (all k must succeed). At k=1: 70%. At k=3: pass@k=91% but pass^k=34%. Takeaway: let Claude retry—success rate improves dramatically.
- **Skill audit:** `/skill-stocktake`—Quick Scan or Full Stocktake. Verdicts: Keep, Improve, Update, Retire, Merge.

Continuous Learning v2.1 is ECC's most innovative feature. It uses atomic "instincts" with confidence scoring:

```
id: prefer-functional-style
trigger: "when writing new functions"
confidence: 0.7    # 0.3=tentative, 0.5=moderate,
                    # 0.7=strong, 0.9=near-certain
domain: "code-style"
scope: project      # or 'global'
project_id: "a1b2c3d4"  # hash of git remote URL
```

The observation pipeline fires on **every** PreToolUse and PostToolUse (hooks = reliable). It scrubs secrets via regex, auto-archives at 10MB, purges after 30 days, and scopes observations to projects via git remote URL hash. When the same instinct appears in 2+ projects with average confidence $\geq$0.8, it auto-promotes to global scope.

3 hook profiles:

Hook	minimal	standard	strict
Session start/end, cost tracking	✓	✓	✓
Quality gates, auto-format, TS check	—	✓	✓
Continuous learning observe	—	✓	✓
tmux + git push reminders	—	—	✓

When it fits: Everyone. Start with the minimal profile (session persistence + cost tracking) and graduate upward.

Caveats: The standard and strict profiles add overhead per tool call (500–2,000 tokens for observation + quality gates). Monitor latency. The continuous learning pipeline requires understanding the observation format before it delivers value.

Alternatives: For just the learning aspect, Self-Improving Agent (alirezarezvani) is simpler. For just the hooks, claude-code-showcase has comparable quality gates.

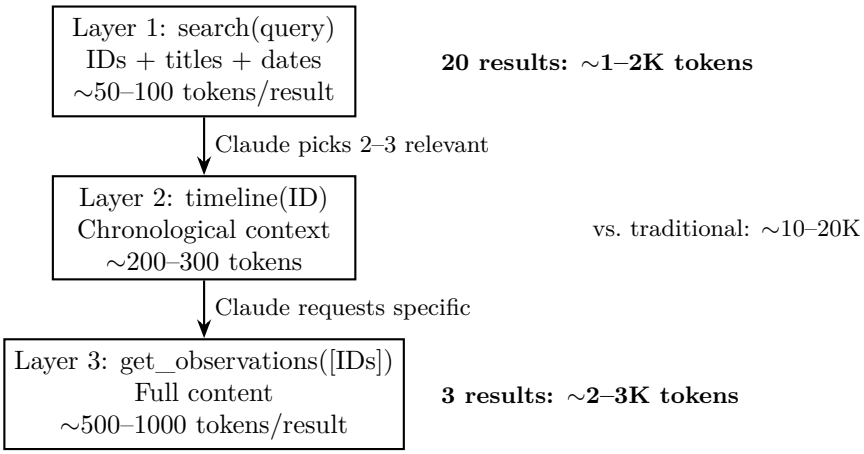
5.2 thedotmack/claude-mem (33.4k stars)

What it is: The most sophisticated memory system in the community. Solves the fundamental problem: Claude forgets everything between sessions.

Progressive disclosure—the core innovation:

Traditional retrieval fetches all content at once. Search for "transport routes," get 20 results, and Claude receives the full content of all 20—typically 10–20K tokens. Most results are irrelevant, but the tokens are spent.

Progressive disclosure solves this with **3 layers**:



Architecture: SQLite is the source of truth (7 tables, FTS5 for keyword search, WAL mode, 256MB mmap). Chroma is a secondary index for semantic search. Each field in an observation (narrative, fact_0, fact_1, etc.) becomes a separate Chroma document—**granular per-field embedding** that matches on individual facts rather than whole observations.

Token savings on two levels:

1. **AI compression:** Raw tool output (∼5,000 tokens) is compressed into structured observations with title, subtitle, narrative, facts (∼500–1,000 tokens).
2. **Progressive disclosure:** Even compressed content is only fetched when relevant.

When it fits: When memory across sessions is critical and you need production-grade retrieval. Best for users who work across many sessions on the same project and frequently need to recall past decisions, discussions, or code changes.

Caveats:

- **AGPL-3.0 license:** Source code must be published if the system is exposed over a network (AGPL §13). For private local use (MCP over stdio), risk is low but non-zero.
- **Bun runtime required:** If your stack is Python/bash, Bun is a new dependency to maintain.
- **ragtime/ subdirectory** uses PolyForm Noncommercial 1.0.0—prohibits all commercial use.
- The "∼10x token savings" claim is from claude-mem's own benchmarks, not independently verified.

Alternatives: Build a 3-layer retrieval system yourself against an existing vector database (e.g. Qdrant). Simpler, no license issues, but requires implementation work.

5.3 alirezarezvani/claude-skills (2.3k stars)

What it is: 160+ skills, with the standout being the **Self-Improving Agent**—a system that lets Claude learn from its own patterns over time.

Self-Improving Agent commands:

/si:review Scans MEMORY.md for recurring patterns. Identifies which observations keep appearing.

/si:promote Graduates confirmed patterns from MEMORY.md (temporary) to CLAUDE.md (permanent). This is the key action: important knowledge that would otherwise be lost when MEMORY.md overflows is promoted to a durable instruction.

/si:extract Converts patterns into standalone skills that can be reused across projects.

/si:status Memory capacity dashboard.

When it fits: When MEMORY.md fills up with repeated observations that should be permanent instructions. The promotion pipeline solves a real gap: knowledge that gets rediscovered every session instead of being codified once.

Caveats: The promotion process is aggressive—review what it promotes to CLAUDE.md. A wrong pattern codified as a permanent instruction is worse than no instruction.

Alternatives: ECC Continuous Learning v2.1 is more sophisticated (confidence scoring, project scoping) but heavier to set up.

5.4 Jeffallan/claude-skills (5.6k stars)

What it is: 66 skills focused on development workflow. The standout is `/common-ground`.

/common-ground: Surfaces Claude's assumptions about the project. When starting work, Claude may silently assume things about the tech stack, coding conventions, or project goals. This skill forces Claude to explicitly state its assumptions so you can correct them before work begins.

Also includes Jira and Confluence workflow integrations for enterprise teams.

When it fits: At the start of a new project or when onboarding Claude to an unfamiliar codebase. The assumption-surfacing pattern prevents wasted work from wrong assumptions.

Caveats: Jira/Confluence integrations assume enterprise tooling. The standalone skills are more broadly useful.

Alternatives: You can achieve similar assumption-surfacing by adding a rule to CLAUDE.md: "Before starting any new task, state your assumptions and wait for confirmation."

5.5 sickn33/antigravity-awesome-skills (21.6k stars)

What it is: The largest skills collection—1,232 skills across all categories.

When it fits: As a browsing catalog. When you need a skill for a specific niche (e.g. Kubernetes deployment, Terraform plans, GraphQL schema generation), search this repo first.

Caveats: Quality varies enormously. Many skills are trivial wrappers. No curation or quality guarantee. Always read a skill's source before installing it—some may contain unsafe patterns.

Alternatives: For curated, high-quality skills, start with ECC or the official plugins. Use Antigravity Awesome only when you need something specific that doesn't exist elsewhere.

5.6 VoltAgent/awesome-claude-code-subagents

What it is: Collection of 127+ subagent configurations across 10 domains, each with isolated context windows.

When it fits: When designing multi-agent architectures. The value is in the design patterns, not the specific configurations (which need adaptation to your stack).

Caveats: Many subagent configs are generic. The interesting ones are domain-specific (e.g. security-focused, data-pipeline-focused). Copy the pattern, adapt the specifics.

Alternatives: NicholasSpisak's 9 personas (see Workflow section) are more focused and opinionated.

5.7 makenotion/claude-code-notion-plugin

What it is: Official plugin from Notion for Claude Code. Adds 4 skills and 10 slash commands on top of the built-in Notion MCP.

Skills:

Knowledge Capture Auto-saves important findings to Notion during a session.
Meeting Intelligence Structures meeting notes and action items.
Research Documentation Organizes research findings in databases.
Spec to Implementation Converts Notion specifications into code.

When it fits: When Notion is your primary organizational tool and you want deeper integration than the basic MCP provides.

Caveats: Known Cowork-mode bug (Issue #18680): tools disappear after first prompt. Workaround: restart session. Notion MCP cannot create views (Kanban, Calendar)—views must be created manually in Notion's UI.

Alternatives: The built-in Notion MCP handles 80% of use cases. This plugin adds the remaining 20%.

5.8 YouMind-OpenLab/nano-banana-pro-prompts

What it is: 10,000+ curated prompts for Google Gemini's image generation model (Gemini 3 Pro Image / Nano Banana Pro).

When it fits: When generating visual content—diagrams, infographics, illustrations, concept art—using Gemini's native image generation. The prompts are structured with subject, medium, style, lighting, composition, and palette.

Caveats: Gemini-specific. These prompts do not transfer well to other image models (DALL-E, Midjourney). Requires Google Gemini API access.

Alternatives: For Claude-native visual generation, use Mermaid (diagrams) or the diagrams Python library (infrastructure). For general image generation, use whatever model you have API access to.

5.9 tfriedel/claude-office-skills

What it is: Skills for working with Microsoft Office formats: PPTX, DOCX, XLSX, PDF.

When it fits: When Claude needs to generate or modify Office documents. Useful for automated report generation, presentation building, or spreadsheet manipulation.

Caveats: Heavy dependency chain (python-pptx, python-docx, openpyxl). The skills assume specific Python libraries are installed. Quality of generated documents varies—complex formatting is difficult to get right programmatically.

Alternatives: For PDF: LaTeX (which this report uses). For presentations: Mermaid diagrams exported to slides. For spreadsheets: direct CSV/JSON manipulation is often simpler.

5.10 mksglu/context-mode (2.9k stars)

What it is: A tool that compresses context to reduce token usage. It pre-processes files and conversation history to be more token-efficient before sending to Claude.

When it fits: When working with large codebases where many files need to be in context simultaneously. It compresses boilerplate and repetitive code patterns.

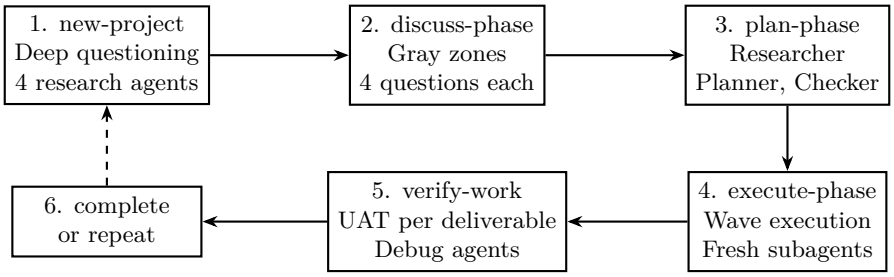
Caveats: Cannot intercept MCP tools—only works with direct file operations. Less relevant for setups that already use selective file loading.

Alternatives: Progressive disclosure (claude-mem) solves the same problem more fundamentally. Strategic model selection (Haiku for exploration, Opus for critical work) is a simpler approach.

6 Repository Catalog — Workflow & Architecture

6.1 gsd-build/get-shit-done [GSD] (26.1k stars)

What it is: The most comprehensive workflow framework for Claude Code. Solves context rot by decomposing large tasks into small, independently-executable plans with fresh context per task.



STATE.md (<150 lines): A slim pointer file containing Project Reference, Current Position (phase/status/ASCII progress bar), Performance Metrics (velocity, trend), Accumulated Context (decisions, blockers), and Session Continuity (timestamp, resume path).

Wave execution: Plans have dependencies defined in PLAN.md frontmatter:

```
---
phase: 01-auth
plan: 03
wave: 2
depends_on: [plan-01, plan-02]
files_modified: [src/auth/login.ts]
autonomous: true
---
```

Wave number: `wave = max(dependency_waves) + 1`. Within a wave: parallel. Between waves: sequential. Spot-check after each wave. Config: `max_concurrent_agents: 3`.

Fresh context per task: Each plan is executed by a spawned subagent with 200K clean context. The subagent loads: PLAN.md, config.json, STATE.md (paths only), CLAUDE.md, skills. It does **not** load: session history, full project context, decision logs. The orchestrator stays at 10–15% context utilization.

4 deviation rules:

1. Bug → auto-fix
2. Missing Critical (validation, auth) → auto-add
3. Blocking (dependencies, imports) → auto-fix
4. Architectural → **STOP** and ask user

When it fits: Large features that span multiple files and take more than one session. Projects where context rot has caused visible quality degradation. Teams where multiple people need to understand the work plan.

Caveats: Heavy. The full GSD workflow (new-project → discuss → plan → execute → verify) is overkill for small tasks. The sweet spot is adopting **principles** without installing the framework: fresh context per task (already possible via Task tool), deviation rules (add to CLAUDE.md), context monitor (a simple hook).

Alternatives: claude-quickstarts' Autonomous Coding Agent for a simpler two-agent pattern. Or just use Task/Agent tools directly with discipline.

6.2 ChrisWiles/claude-code-showcase (5.5k stars)

What it is: Practical hooks, quality gates, and CI/CD workflows for Claude Code. Where GSD provides the strategy, showcase provides the tactical implementation.

Skill Evaluation Hooks—3 layers:

1. **skill-eval.sh**—Bash wrapper on UserPromptSubmit. Pipes stdin to JavaScript.
2. **skill-eval.js** (∼300 lines)—7 signal types with scoring: Keywords (weight 2), Keyword patterns (3), Path patterns (4), Directory mapping (5), Intent patterns (4), Content patterns (3), Context patterns (2). Minimum confidence 3, top 5 results, exclude patterns.
3. **skill-rules.json**—19 skills defined with triggers, excludePatterns, priority, relatedSkills.

This system automatically selects the right skill based on the user's prompt. It is essentially a local, rule-based routing layer.

Quality Gate Hooks:

- **PreToolUse (Edit|Write):** Branch protection—blocks edits on main.
- **PostToolUse (Edit|Write):** Auto-format (Prettier), auto-install (npm), auto-test, type-check (tsc –noEmit). All non-blocking—they provide feedback but don't stop work.

4 GitHub Actions workflows:

Workflow	Cadence	Function
Docs Sync	Monthly	30 max-turns, 60 min timeout
Code Quality	Weekly (Sunday)	3 random src/ dirs, 35 turns, 45 min
Dependency Audit	Biweekly	npm outdated + audit, reverts on test failure
PR Code Review	On PR creation	Automatic review

When it fits: TypeScript/JavaScript projects with established CI/CD. The quality gates are directly usable. The skill evaluation pattern is worth studying even if you don't use JavaScript.

Caveats: JavaScript/Node-centric. The PostToolUse hooks assume npm, Prettier, and TypeScript. Adapt for other stacks.

Alternatives: For branch protection alone, a 5-line bash hook is sufficient (see Implementation Plan). For auto-formatting, integrate your stack's formatter (Black for Python, rustfmt for Rust, etc.).

6.3 NicholasSpisak/claude-code-subagents

What it is: 9 specialized subagent personas, each with a defined focus area and priority hierarchy.

Persona	Priority order
backend-reliability	Reliability > Security > Performance > Readability
frontend-ux	UX > Accessibility > Performance > Consistency
systems-architect	Scalability > Maintainability > Security > Performance
security-threat	Security > Compliance > Integrity > Availability
performance-optimizer	Latency > Throughput > Resource usage > Scalability
code-analyzer	Readability > Correctness > Consistency > Efficiency
refactoring-expert	Modularity > Testability > Readability > DRY
qa-test	Coverage > Reliability > Maintainability > Speed
technical-mentor	Understanding > Best practice > Context > Progression

Design principle: Specialization over generalization. Each agent knows a lot about its domain and little about everything else. Combining 3–4 agents provides broader coverage than a single generalist.

When it fits: Code review and architectural decisions where multiple perspectives matter. Run 3 specialized agents in parallel for a comprehensive review.

Caveats: The specific context budgets (50%/70%/85%) mentioned in some discussions could not be verified against the actual repository (ISS-002).

Alternatives: The official code-review plugin provides 5 parallel review agents. VoltAgent has 127+ configurations. Both are less opinionated about priority hierarchies.

6.4 shanraisshan/claude-code-best-practice

What it is: A curated collection of best practices for Claude Code usage, gathered from community experience.

When it fits: Quick reference for common patterns and anti-patterns. Good onboarding material.

Caveats: Community-maintained, not official. Practices may not reflect the latest Claude Code features.

Alternatives: ECC's guides are more comprehensive. Official docs are authoritative.

6.5 teambrilliant/claude-research-plan-implement [RPI]

What it is: Research-Plan-Implement workflow. A lighter alternative to GSD focused on three phases: research the codebase, plan the changes, implement them.

When it fits: Medium-complexity tasks where GSD's 6-phase workflow is too heavy but you still want structured planning. Good for feature additions that touch 3–5 files.

Caveats: Less mature than GSD. Fewer safeguards against context rot.

Alternatives: GSD for large tasks. Direct Claude Code for small tasks. RPI fills the middle ground.

6.6 dontriskit/awesome-ai-system-prompts

What it is: Collection of system prompts from various AI tools, including Claude Code's internal system prompt.

When it fits: Understanding how Claude Code works internally. The system prompt reveals Claude's built-in behaviors, restrictions, and patterns. Useful for debugging unexpected behavior.

Caveats: System prompts change with updates. What you see may not match the current version.

Alternatives: Official documentation for current behavior.

6.7 lbjlaq/Antigravity-Manager

What it is: A proxy that routes AI requests through free model providers, potentially giving access to Opus 4.6 without API costs.

When it fits: If API budget is a hard constraint and you need to experiment with expensive models.

Caveats: CC-BY-NC-SA license (non-commercial). Free model providers may have rate limits, quality issues, or disappear without notice. Not suitable for production or sensitive work.

Alternatives: Anthropic's own API with Sonnet (significantly cheaper than Opus) for most work. Budget management via cost monitoring.

6.8 alibaba/page-agent

What it is: AI-powered DOM manipulation tool from Alibaba. Claude can interact with web pages by understanding and modifying their structure.

When it fits: Web scraping, automated testing, or web application development where Claude needs to understand page structure.

Caveats: Enterprise-oriented. Heavy dependency on Alibaba's ecosystem. Not well-suited for personal use.

Alternatives: Playwright or Puppeteer for web automation. Claude's built-in Computer Use for visual web interaction.

7 Repository Catalog — Visual Tools

7.1 mermaid-js/mermaid (86.5k stars)

What it is: Converts text-based descriptions into SVG diagrams. The most widely adopted diagramming tool in the developer ecosystem.

Supported diagram types: Flowcharts, sequence diagrams, Gantt charts, class diagrams, state diagrams, entity-relationship, Git graphs, C4 architecture diagrams, mindmaps, timelines.

Example:

```
graph TD
    A[User] --> B[Claude Code]
    B --> C[Hooks]
    B --> D[Skills]
    B --> E[MCP Servers]
    C --> F[save_checkpoint.py]
    D --> G[/commit]
    E --> H[Notion]
```

When it fits: Always. Mermaid renders natively in GitHub READMEs, PR descriptions, and Issues. Diagrams are versionable (plain text). Claude generates Mermaid syntax reliably.

Caveats: Complex diagrams with many nodes become unreadable. Mermaid's layout algorithm has limited manual positioning control. For precise layouts, use TikZ or diagrams.

Alternatives: TikZ for print-quality diagrams (what this report uses). diagrams (Python) for infrastructure-specific diagrams. PlantUML for UML-heavy projects.

7.2 mingrammer/diagrams (42.1k stars)

What it is: Python library that generates infrastructure diagrams from code. Instead of drawing, you write Python:

```python
from diagrams import Diagram, Cluster
from diagrams.onprem.container import Docker
from diagrams.onprem.network import Nginx

with Diagram("Infrastructure"):
    with Cluster("VPS"):
        nginx = Nginx("Traefik")
        app = Docker("Webapp")
        nginx >> app
```

Supported providers: AWS, GCP, Azure, Kubernetes, On-Premises, Generic, C4. Uses Graphviz for rendering.

When it fits: Infrastructure documentation. The code-as-diagram approach means diagrams are versionable, reproducible, and reviewable in PRs. Python-native—fits Python-based stacks.

Caveats: Requires Graphviz (`apt install graphviz`). Output is PNG or PDF, not interactive SVG. The library's icon set is infrastructure-focused—not suitable for general-purpose diagramming.

Alternatives: Mermaid for quick inline diagrams. TikZ for print-quality output. draw.io for interactive editing.

7.3 antvis/Infographic (4.6k stars)

What it is: TypeScript library for declarative infographic generation. 200 built-in templates. AI-optimized—Claude can generate configuration objects directly. Streaming render. SVG output.

When it fits: Generating status reports, data visualizations, or presentation-quality infographics programmatically. The templates give quick results without design expertise.

Caveats: TypeScript dependency (doesn't match Python/bash stacks). Still relatively new (4.6k stars). The "AI-optimized" claim means the configuration format is LLM-friendly, not that it uses AI internally.

Alternatives: Mermaid for simple diagrams. Matplotlib/Seaborn for data visualization in Python. Gemini image generation (Nano Banana Pro) for creative visuals.

7.4 federicodeponte/opendraft (55 stars)

What it is: A 19-agent pipeline that generates academic research drafts (20,000+ words). Uses CrossRef, OpenAlex, Semantic Scholar, and arXiv for citations. Export to PDF, Word, LaTeX.

When it fits: Academic research projects where you need a structured literature review or research draft with proper citations. Uses the same academic APIs as many research scripts.

Caveats: Very low adoption (55 stars). Unproven at scale. The 19-agent pipeline is computationally expensive. Quality of generated academic text needs verification.

Alternatives: Manual research with `scripts/research.py` + Claude for writing. LaTeX for formatting. The advantage of OpenDraft is automation; the risk is hallucinated citations.

7.5 Shubhamsaboo/awesome-llm-apps (100k stars)

What it is: The largest curated collection of LLM applications. Covers RAG tutorials, knowledge graph RAG, multi-agent systems, and applications across all categories.

When it fits: Inspiration and reference. When building a new LLM-powered feature, search this repo for existing implementations to learn from.

Caveats: Breadth over depth. Most examples are starting points, not production code. Quality varies.

Alternatives: claude-cookbooks for Claude-specific recipes. The Python Agent SDK for Claude-native agent building.

7.6 openclaw/openclaw

What it is: A personal AI platform with heartbeat daemon, hybrid search, temporal decay, and auto-save. The most architecturally ambitious personal AI system found in this survey.

Key principles worth studying:

- **Heartbeat daemon:** Runs every 30 minutes (not just daily cron). Checks for pending tasks, triggers proactive actions.
- **Hybrid search:** Vector 70% + BM25 30% for retrieval. Combines semantic understanding with keyword precision.

- **Temporal decay:** Newer information scores higher. Prevents stale data from dominating results.
- **Auto-save before compaction:** Saves important knowledge before context compression occurs.

When it fits: As a design reference. The principles above are gold. The actual codebase is 430K lines of Node.js—too heavy to install.

Caveats: Estimated operational cost: $2–5K/month. Node.js stack. Not designed for personal single-user systems.

Alternatives: Steal the principles, build simpler implementations. Heartbeat = cron job. Hybrid search = Qdrant configuration. Temporal decay = query-time score adjustment. Auto-save = PreCompact hook.

8 Gap Analysis — Yggdra vs. The Ecosystem

8.1 What Yggdra already has (validated by the ecosystem)

Yggdra feature	Ecosystem equivalent	Assessment
`projects/*/NOW.md`	GSD STATE.md, RPI thoughts/	**Validated.** GSD adds performance metrics and ASCII progress bar.
`projects/*/CONTEXT.md`	ECC per-project CLAUDE.md, GSD PROJECT.md	**Validated.** GSD further splits into REQUIREMENTS.md and ROADMAP.md.
save_checkpoint + load	ECC session hooks, claude-mem hooks	**Validated.** ECC adds HTML comment markers for idempotent updates.
episodes.jsonl	claude-mem observations + session_summaries	**Primitive.** claude-mem has 7 tables, FTS5, granular observations with confidence scoring.
4 hook events used	17 official events available	**Partial.** Missing UserPromptSubmit, PostToolUse, SubagentStart/Stop.
`ctx` (Qdrant search)	claude-mem 3-layer, ECC search-first	**Partial.** Missing progressive disclosure and FTS5 keyword search.
CLAUDE.md <200 lines	Best practice, 2% skill budget	**Validated.**
MEMORY.md	claude-mem SQLite, SI Agent, ECC instincts	**Partial.** Missing auto-promotion, confidence scoring, project scoping.
Bash-first approach	ECC: CLI-wrappers > MCP	**Reinforced.** Yggdra's approach confirmed by community consensus.

8.2 What Yggdra is missing (prioritized)

1. **Quality gate hooks** (high priority, low complexity)—Branch protection + auto-format.
2. **Context monitor hook** (high priority, low complexity)—Warn at 35%, critical at 25%.
3. **Official plugins** (high priority, zero risk)—hookify, security-guidance, commit-commands.
4. **Hybrid search** (high priority, medium complexity)—Qdrant supports BM25 + vector natively. Configuration change, not new code.
5. **Progressive disclosure** (high priority, high complexity)—3-layer retrieval. Requires baseline measurement first.
6. **Auto-promotion** (medium priority, low complexity)—Self-Improving Agent: MEMORY.md → CLAUDE.md.
7. **Fresh context per task** (medium priority, low complexity)—Already possible via Task tool.

8. **Observation extraction** (medium priority, high complexity)—Raw tool output → structured observations.
9. **Instinct/confidence scoring** (medium priority)—ECC v2.1 project-scoped instincts.
10. **Wave execution** (low priority)—GSD dependency mapping. Overkill for now.

8.3 What Yggdra does better

1. **Multi-project isolation.** CONTEXT.md + NOW.md per project. GSD has similar features but operates per-project—Yggdra's model handles many projects simultaneously with a single orchestration layer.
2. **Episodic memory via cheap LLM.** `episodes.jsonl` + Groq distillation. No community solution has automatic session distillation via a cheap LLM (claude-mem uses a full Claude instance as worker).
3. **Checkpoint safeguards.** 10-min throttle, 80KB cap, hash-dedup. Protects against duplication and overwhelming state files. Not observed in ECC or claude-mem.
4. **Kill conditions.** An explicit anti-pattern in CLAUDE.md: "No new service/cron without a defined condition for removal." Not identified in any evaluated repository.
5. **Production-grade vector DB.** Qdrant with 84K vectors and 7 collections. Community uses flat files (ECC) or SQLite + Chroma (claude-mem). Yggdra's vector infrastructure is more mature.

9 Recommendations

9.1 Classification summary

Based on the individual repository evaluations, the following classification emerges:

Adopt (use now): VS Code + Extension, Notion MCP (built-in), Anthropic Academy, official plugins (hookify, security-guidance, commit-commands), quality gate hooks, context monitor hook, Mermaid.

Trial (experiment actively): ECC minimal profile, GSD fresh-context principle, claude-mem progressive disclosure, Self-Improving Agent, Notion Plugin, diagrams (Python), Nano Banana Pro.

Assess (watch): ECC Continuous Learning, AntV Infographic, Antigravity Manager, Context Mode, VoltAgent, skill evaluation hooks, OpenDraft, GitHub Actions workflows, GSD wave execution.

Hold (park): Cursor, Google Antigravity (browser), skill marketplaces, Claude Office Skills, OpenSandbox, LaTeX2AI, benbrastmckie/nvim.

9.2 Implementation plan

Sequenced by dependency and impact. Each step is independent of timing.

9.2.1 Step 1: Foundation

1. Install VS Code + Claude Code extension on PC
2. `git clone` Yggdra repo
3. Configure `-add-dir` to VPS copy or symlink
4. Verify that CLAUDE.md, hooks, and skills load correctly

9.2.2 Step 2: Official plugins + hooks

5. Install hookify, security-guidance, commit-commands
6. Implement branch protection hook (PreToolUse):

```
# Branch protection - denies edits on main
branch=$(git branch --show-current)
if [ "$branch" = "main" ]; then
  echo '{"hookSpecificOutput":{
    "hookEventName":"PreToolUse",
    "permissionDecision":"deny",
    "permissionDecisionReason":"Create a feature branch first."
  }}'
  exit 2
fi
```

7. Implement context monitor hook (PostToolUse)

9.2.3 Step 3: Community trials

8. Self-Improving Agent: test `/si:review` and `/si:promote`
9. Nano Banana Pro Prompts
10. Notion Plugin
11. `pip install diagrams && apt install graphviz`

9.2.4 Step 4: Workflow

12. Test fresh-context-per-task (via Task tool)
13. Add deviation rules to CLAUDE.md
14. Evaluate: does it improve quality vs. current workflow?

9.2.5 Step 5: Memory

15. Measure baseline token consumption via `cost_guardian.py`
16. Evaluate claude-mem: is AGPL §13 + Bun acceptable?
17. Alternative: prototype 3-layer retrieval against Qdrant
18. Enable hybrid search in Qdrant (BM25 + vector)

9.2.6 Ongoing

- Anthropic Academy courses (Agent Skills → Claude Code in Action → MCP Advanced)
- ECC minimal profile (`ECC_HOOK_PROFILE=minimal`)
- Weekly skill-stocktake
- Monthly reassessment of Trial and Assess items

10 Maintenance & Kill Conditions

10.1 Cadences

Cadence	Action
Daily	Checkpoint + episodes (existing, automatic)
Weekly	`/skill-stocktake` audit of installed skills
Monthly	Reassess: have Assess items matured to Trial?
Quarterly	Full report update: new sources, star counts, trends

10.2 Kill conditions

Every new tool must have a defined removal condition **before** installation:

- **ECC:** Remove if hooks add >5 sec latency per tool call, or user manually disables them >3 times/week.
- **claude-mem:** Remove if AGPL §13 is incompatible with setup, or Bun dependency creates maintenance burden >1 hour/month.
- **Skill evaluation:** Remove if false-positive rate >30%.
- **Notion Plugin:** Remove if it crashes Cowork-mode or adds >5 tools to context.
- **Any trial tool:** Remove if not used in 30 days.

10.3 Cost monitoring

Prerequisite: Measure baseline token consumption per session via `cost_guardian.py` **before** installing new tools. Without a baseline, you cannot measure the effect of new tooling.

11 Open Questions

1. **AGPL §13 evaluation**
   claude-mem's AGPL-3.0 license requires source code disclosure if the system is exposed over a network. For private local use (MCP over stdio), the risk is low but non-zero. The `ragtime/` subdirectory uses PolyForm Noncommercial 1.0.0—prohibits all commercial use.
2. **Bun dependency**
   claude-mem requires Bun runtime. Adding a new runtime means security updates, version conflicts, and debug overhead.
3. **Progressive disclosure MVP**
   A Qdrant endpoint returning IDs + titles + scores (layer 1). Success criterion: retrieval uses <2K tokens for scanning 20 results (vs. current ∼10–20K).
4. **Token cost monitoring for subagents**
   None of the evaluated repositories have a good solution for measuring real token cost per session including subagent overhead.

12 Known Issues

ID	Description	Status
ISS-001	Agent subprocess stalled during PRD+Skills absorption. Manually replaced with direct WebFetch. Possible cause: rate-limiting or context overflow.	Workaround
ISS-002	NicholasSpisak context budgets (50/70/85%) not verifiable against repository. Data retained with caveat.	Acknowledged

13 Complete Link Catalog

Star counts as of March 8, 2026.

13.1 Official Anthropic

- `anthropics/skills` (86.9k)—Official skills standard
- `anthropics/claude-code` (75.1k)—Claude Code + 13 plugins
- `anthropics/claude-cookbooks` (34.4k)—Practical recipes
- `anthropics/courses` (19.2k)—Anthropic Academy
- `anthropics/claude-quickstarts` (15.1k)—Starter projects
- `anthropics/claude-code-action` (6.1k)—GitHub Actions
- `anthropics/claude-agent-sdk-python` (5.2k)—Python Agent SDK
- `https://agentskills.io`—Skills standard docs
- `https://anthropic.skilljar.com/`—Anthropic Academy (13 free courses)
- `https://code.claude.com/docs/en/skills`—Official skills reference
- `https://code.claude.com/docs/en/hooks`—Official hooks reference

13.2 Community — Skills & Memory

- `affaan-m/everything-claude-code` (65.8k)—65 skills, 16 agents, 3 guides
- `thedotmack/claude-mem` (33.4k)—Memory + progressive disclosure
- `sickn33/antigravity-awesome-skills` (21.6k)—1,232 skills
- `Jeffallan/claude-skills` (5.6k)—66 skills, /common-ground
- `ChrisWiles/claude-code-showcase` (5.5k)—Hooks, quality gates
- `mksglu/context-mode` (2.9k)—Context compression
- `alirezarezvani/claude-skills` (2.3k)—Self-Improving Agent
- `VoltAgent/awesome-claude-code-subagents`—127+ subagents
- `makenotion/claude-code-notion-plugin`—Official Notion plugin
- `YouMind-OpenLab/nano-banana-pro-prompts`—10,000+ Gemini prompts
- `tfriedel/claude-office-skills`—PPTX, DOCX, XLSX, PDF

13.3 Workflow & Architecture

- `gsd-build/get-shit-done` (26.1k)—Spec-driven workflow, wave execution
- `shanraisshan/claude-code-best-practice`—Best practices
- `teambrilliant/claude-research-plan-implement`—RPI workflow
- `NicholasSpisak/claude-code-subagents`—9 specialized personas
- `dontriskit/awesome-ai-system-prompts`—System prompt internals
- `lbjlaq/Antigravity-Manager`—AI account proxy
- `alibaba/page-agent`—AI DOM manipulation
- `openclaw/openclaw`—Personal AI platform

13.4 Visual Tools

- `mermaid-js/mermaid` (86.5k)—Markdown → SVG diagrams
- `mingrammer/diagrams` (42.1k)—Python infrastructure diagrams
- `antvis/Infographic` (4.6k)—AI-optimized infographic generation
- `federicodeponte/opendraft` (55)—19-agent research pipeline
- `Shubhamsaboo/awesome-llm-apps` (100k)—LLM apps collection

13.5 Guides & Articles

- How I Use Every Claude Code Feature (blog.sshh.io)

- Ultimate Claude Code Setup (Medium)
- Claude Code Mobile (happy.engineering)
- SkillsBento—`https://www.skillsbento.com/`
- Skillzwave—`https://skillzwave.ai/docs/`
- Paks—`https://paks.stakpak.dev/`